discoCVI

Density-based Internal Score for Clusterings with nOise

R Package: Translation, Implementation & Experimental Validation

Author
Amin Entezari
amin_entezari@outlook.com

Supervisor
Davide Chicco
davidechicco@davidechicco.it

Repository
github.com/aminentezari/discoCVI

```
devtools::install_github(
"aminentezari/discoCVI")
```

Reference Implementation
Beer, Krieger, Weber,
Ritzert, Assent, Plant (2025)
Internal Evaluation of Density-Based Clusterings with Noise
arXiv:2503.00127
Presented at ICLR 2026

Abstract. This report documents the faithful line-by-line translation of the Python DISCO reference implementation into R for CRAN publication as the `discoCVI` package. Three implementation bugs were identified and corrected. Numerical verification across **eight benchmark datasets** — covering rings, crescents, spirals, mixed-density clusters, and Gaussian blobs — confirms that the R implementation produces results within IEEE 754 floating-point precision ($|\Delta| < 10^{-14}$) of the Python reference. The package is available at `https://github.com/aminentezari/discoCVI`.

April 30, 2026

Contents

1 Introduction **3**

2 Algorithm **3**
2.1 Problem Statement . . . 3
2.2 Step 1: Core Distance . . . 3
2.3 Step 2: Mutual Reachability Distance . . . 3
2.4 Step 3: DC-Distance via MST . . . 4
2.5 Step 4: Scoring Cluster Points . . . 4
2.6 Step 5: Scoring Noise Points . . . 4
2.7 Overall DISCO Score . . . 4
2.8 Algorithmic Pipeline . . . 4

3 R Implementation **5**
3.1 Translation Strategy . . . 5
3.2 Installation . . . 5
3.3 Quick Start . . . 5

4 Implementation Bugs Discovered and Fixed **5**
4.1 Bug 1 — Core Distance Off-by-One . . . 6
4.2 Bug 2 — MST Tie-Breaking Difference . . . 6
4.3 Bug 3 — K-Means Cross-Language Non-Determinism . . . 6
4.4 Bug Summary . . . 7

5 Experimental Validation **7**
5.1 Datasets and Hyperparameters . . . 7
5.2 Numerical Results . . . 7
5.3 Precision Interpretation . . . 9
5.4 Dataset Visualisations . . . 9
5.5 Cross-Dataset Summary . . . 12

6 API Reference **12**
6.1 `disco_score` . . . 12
6.2 `disco_samples` . . . 13
6.3 `compute_dc_distances` . . . 13
6.4 Utility Functions . . . 13

7 Score Interpretation **13**

8 Conclusion **14**

List of Figures

1 Concentric Circles ($n = 300$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN correctly separates both rings (score 0.690); K-Means cuts through them with a vertical boundary (score 0.027). . . . 9
2 Two Moons ($n = 1000$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN follows both crescents (score 0.818); K-Means splits linearly (score 0.283). . . . 10

3 3-Spiral ($n = 312$, $\varepsilon = 2.0$, $k = 3$, $\mu = 5$). DBSCAN traces the spirals (score 0.584); K-Means actively harms the structure (score -0.0014). 10
4 Complex9 ($n = 3031$, $\varepsilon = 15$, $k = 9$, $\mu = 5$). Mixed convex and non-convex clusters. DBSCAN (0.421) outperforms K-Means (0.055). 10
5 Complex8 ($n = 2551$, $\varepsilon = 10$, $k = 8$, $\mu = 5$). High within-cluster density variation limits both algorithms. 11
6 Dartboard1 ($n = 1000$, $\varepsilon = 0.08$, $k = 4$, $\mu = 5$). Four concentric rings: highest DBSCAN score (0.874); K-Means pizza-slice boundary is actively harmful (score -0.003). 11
7 Blobs ($n = 450$, $\varepsilon = 0.5$, $k = 3$, $\mu = 5$). Convex Gaussian clusters: both algorithms score above 0.85, confirming DISCO rewards correct clustering regardless of algorithm type. 11
8 Diagonal Blobs ($n = 600$, $\varepsilon = 0.8$, $k = 3$, $\mu = 5$). Elongated but linearly separable clusters: both algorithms perform well. 12

1. Introduction

DISCO (Density-based Internal Score for Clusterings with nOise) is a cluster validity index (CVI) introduced by Beer et al. [2025] that evaluates clustering quality *without requiring ground truth labels*. Unlike classical CVIs such as the Silhouette Coefficient [Rousseeuw, 1987] and Davies-Bouldin Index [Davies and Bouldin, 1979], DISCO:

- Handles **arbitrary-shaped clusters** using *density-connectivity distance* (dc-distance);
- Explicitly evaluates **noise point quality** via two sub-scores (p_{sparse} and p_{far});
- Returns a bounded, interpretable score in $[-1, 1]$.

Why DISCO matters. Traditional CVIs can rank an incorrect algorithm *higher* than the correct one on non-convex data. On concentric circles, Silhouette assigns -0.16 to DBSCAN (correct) and $+0.37$ to K-Means (wrong). DISCO reverses this: DBSCAN scores $+0.69$ and K-Means scores $+0.027$.

This report documents:

1. The mathematical algorithm (§2);
2. The R translation approach (§3);
3. Three bugs found and fixed (§4);
4. Numerical validation across eight datasets (§5);
5. The public API reference (§6).

2. Algorithm

2.1 Problem Statement

Given $\mathbf{X} = \{x_1, \ldots, x_n\} \subset \mathbb{R}^p$ and a clustering $\mathcal{C} = \{C_1, \ldots, C_K\}$ with optional noise $\mathcal{N} = \{x_i : \ell_i = -1\}$, DISCO assigns $\rho(x_i) \in [-1, 1]$ to every point and returns the mean as the overall score.

2.2 Step 1: Core Distance

$$\kappa_\mu(x_i) = d\big(x_i,\ x_i^{(\mu-1)}\big)$$

where $x_i^{(k)}$ is the k-th nearest neighbour of x_i.

2.3 Step 2: Mutual Reachability Distance

$$d_{\text{reach}}(x_i, x_j) = \max\big(\kappa_\mu(x_i),\ \kappa_\mu(x_j),\ d(x_i, x_j)\big)$$

2.4 Step 3: DC-Distance via MST

1. Build graph G with weights d_{reach}.
2. Compute MST T via Prim's algorithm.
3. $d_{\text{DC}}(x_i, x_j) = \max_{e \in \text{path}_T(i,j)} w(e)$

2.5 Step 4: Scoring Cluster Points

$$a(i) = \frac{1}{|C_{c_i}| - 1} \sum_{\substack{j \in C_{c_i} \\ j \neq i}} d_{\text{DC}}(x_i, x_j) \tag{1}$$

$$b(i) = \min_{c \neq c_i} \frac{1}{|C_c|} \sum_{j \in C_c} d_{\text{DC}}(x_i, x_j) \tag{2}$$

$$\rho_{\text{cluster}}(x_i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \tag{3}$$

2.6 Step 5: Scoring Noise Points

$$p_{\text{sparse}}(x_n) = \min_c \frac{\kappa_\mu(x_n) - M_c}{\max(\kappa_\mu(x_n), M_c)}, \quad p_{\text{far}}(x_n) = \min_c \frac{d_{\min,c}(x_n) - M_c}{\max(d_{\min,c}(x_n), M_c)}$$

$$\rho_{\text{noise}}(x_n) = \min\big(p_{\text{sparse}}(x_n),\ p_{\text{far}}(x_n)\big)$$

2.7 Overall DISCO Score

$$\boxed{\text{DISCO}(\mathbf{X}, \mathcal{C}) = \frac{1}{n} \sum_{i=1}^{n} \rho(x_i)} \quad \text{where} \quad \rho(x_i) = \begin{cases} \rho_{\text{cluster}}(x_i) & x_i \notin \mathcal{N} \\ \rho_{\text{noise}}(x_i) & x_i \in \mathcal{N} \end{cases}$$

2.8 Algorithmic Pipeline

Step	R Function	Operation
1	`calculate_reachability_distance`	Core distances + mutual reachability matrix
2	`get_mst_edges`	Prim's MST on mutual reachability graph
3	`extract_dc_distances_from_mst`	BFS minimax-path extraction
4	`compute_dc_distances`	Full $n \times n$ DC-distance matrix
5	`p_cluster`	Silhouette-style scores for cluster points
6	`p_noise`	Sparsity + remoteness scores for noise
7	`disco_samples`	Per-point DISCO scores
8	`disco_score`	Mean over all points

3. R Implementation

3.1 Translation Strategy

The `discoCVI` R package is a **line-by-line translation** of `disco.py` + `dctree.py`. The entire package lives in a single file `R/disco.R`:

Section	Python source	Key R functions
DC-Distance	`dctree.py`	`calculate_reachability_distance`, `get_mst_edges`, `extract_dc_distances_from_mst`, `compute_dc_distances`
DISCO Metric	`disco.py`	`disco_score`, `disco_samples`, `p_cluster`, `p_noise`
Utilities	—	`make_circles`, `make_moons`, `make_blobs`, `plot_disco_scores`, `compare_clusterings`

3.2 Installation

```r
# Install from GitHub
devtools::install_github("aminentezari/discoCVI")
library(discoCVI)
```

3.3 Quick Start

```r
library(discoCVI)
library(dbscan)

# Generate data
data      <- make_circles(n_samples = 300, noise = 0.05,
                          factor = 0.4, random_state = 42)
X         <- data$X

# Cluster
db_labels <- dbscan(X, eps = 0.2, minPts = 5)$cluster - 1L
km_labels <- kmeans(X, centers = 2, nstart = 10)$cluster - 1L

# Evaluate
disco_score(X, db_labels)   # -> 0.6900921813 (DBSCAN: correct)
disco_score(X, km_labels)   # -> 0.0270091908 (K-Means: wrong shape)

# Per-point inspection
scores <- disco_samples(X, db_labels)
which(scores < 0)           # identify problematic points

# Compare multiple algorithms
compare_clusterings(X, list(DBSCAN = db_labels, KMeans = km_labels))
```

4. Implementation Bugs Discovered and Fixed

4.1 Bug 1 — Core Distance Off-by-One

Location. `calculate_reachability_distance()` and `p_noise()`

Root Cause. Python (`dctree.py`, line 710):

```
reach_dists[i] = np.max(np.partition(eucl_dists[i], k)[:k])
```

`eucl_dists[i]` includes self (distance = 0), so `[:k]` returns $[0, d_1, \ldots, d_{k-1}]$ and the max is d_{k-1} — the $(k-1)$-th nearest neighbour. `FNN::get.knn` excludes self, so `k = min_points` returns the k-th neighbour.

Fix.

```
# BEFORE (wrong)
knn_result <- FNN::get.knn(X, k = min_points)
core_dists <- knn_result$nn.dist[, min_points]

# AFTER (correct)
k          <- min_points - 1L
knn_result <- FNN::get.knn(X, k = k)
core_dists <- knn_result$nn.dist[, k]
```

4.2 Bug 2 — MST Tie-Breaking Difference

Root Cause. Equal-weight edges arise frequently because many distances are floored to the core distance. Python's Prim (`dctree.py` lines 401–438) breaks ties via `np.argmin`; `igraph::mst` uses different ordering, producing a different MST and different dc-distances.

Fix. Replaced `igraph::mst` with an exact R port of Python's Prim:

```
get_mst_edges <- function(dist_matrix) {
  n              <- nrow(dist_matrix)
  nodes_min_dist <- rep(Inf, n);  parent <- rep(1L, n)
  not_in_mst     <- rep(TRUE, n)
  u <- 1L;  nodes_min_dist[u] <- 0;  not_in_mst[u] <- FALSE
  mst_i <- integer(n-1); mst_j <- integer(n-1)
  mst_dist <- numeric(n-1)
  for (step in seq_len(n - 1L)) {
    mask <- not_in_mst & (dist_matrix[u,] < nodes_min_dist)
    if (any(mask)) {
      nodes_min_dist[mask] <- dist_matrix[u, mask]
      parent[mask]         <- u
    }
    cands <- which(not_in_mst)
    u     <- cands[which.min(nodes_min_dist[cands])]
    mst_i[step] <- parent[u];  mst_j[step] <- u
    mst_dist[step] <- nodes_min_dist[u];  not_in_mst[u] <- FALSE
  }
  data.frame(i = mst_i, j = mst_j, dist = mst_dist)
}
```

4.3 Bug 3 — K-Means Cross-Language Non-Determinism

Root Cause. `set.seed(42)` in R and `random_state=42` in Python use different RNG implementations, producing different centroid initialisations.

Fix.

```
pd.DataFrame({"km_label": km.labels_}).to_csv(
    "python_km_labels.csv", index=False)
```

```
km_labels <- as.integer(
  read.csv("python_km_labels.csv")$km_label)
```

4.4 Bug Summary

#	Location	Root Cause	Fix
1	`calculate_reachability_distance`, `p_noise`	Python includes self; FNN excludes self	`k = min_points - 1`
2	`get_mst_edges`	`igraph::mst` tie-breaking differs	Port Python's Prim exactly
3	All experiment scripts	R and Python RNGs differ	Export Python labels; load in R

5. Experimental Validation

5.1 Datasets and Hyperparameters

Dataset	Source	n	K	Shape	DBSCAN ε	μ
Concentric Circles	Generated	300	2	Rings	0.20	5
Two Moons	Generated	1000	2	Crescents	0.20	5
3-Spiral	`3-spiral.arff`	312	3	Spirals	2.0	5
Complex9	`complex9.arff`	3031	9	Mixed	15.0	5
Complex8	`complex8.arff`	2551	8	Mixed	10.0	5
Dartboard1	`dartboard1.arff`	1000	4	Concentric rings	0.08	5
Blobs	Generated	450	3	Gaussian blobs	0.50	5
Diagonal Blobs	Generated	600	3	Elongated diag.	0.80	5

Notes: K = K-Means cluster count; μ = `min_points`; `min_samples` = μ except 3-Spiral (`min_samples`=3).

5.2 Numerical Results

All values shown to 20 decimal places. Diverging digits: **R** vs **Python**.

Concentric Circles $\varepsilon = 0.20$, min_samples=5, $k = 2$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.69009218128208216214	0.69009218128208293930	7.77×10^{-16}
K-Means	0.02700919079440020423	0.02700919079439973586	4.68×10^{-16}

Two Moons $n = 1000$, $\varepsilon = 0.20$, min_samples=5, $k = 2$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.81795983770753455389	0.81795983770753222242	2.33×10^{-16}
K-Means	0.28323957185132986325	0.28323957185132780934	2.05×10^{-16}

3-Spiral $\varepsilon = 2.0$, min_samples=3, $k = 3$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.58365140464647546903	0.58365140464647502494	4.44×10^{-16}
K-Means	−0.00141763585263621011	−0.00141763585263620208	8.03×10^{-18}

Complex9 $\varepsilon = 15.0$, min_samples=5, $k = 9$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.42094645142453540609	0.42094645142454312214	7.72×10^{-16}
K-Means	0.05510967752575094553	0.05510967752573882328	1.21×10^{-15}

Complex8 $\varepsilon = 10.0$, min_samples=5, $k = 8$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.07186876516496862322	0.07186876516500724510	3.86×10^{-15}
K-Means	0.01225747486413241577	0.01225747486413229781	1.18×10^{-16}

Dartboard1 $\varepsilon = 0.08$, min_samples=5, $k = 4$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.87434268608030629721	0.87434268608031251446	6.22×10^{-15}
K-Means	−0.00336367105007065893	−0.00336367105007149723	8.38×10^{-16}

Blobs $\varepsilon = 0.50$, min_samples=5, $k = 3$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.85489531627953585513	0.85489531627953541104	4.44×10^{-16}
K-Means	0.85756918340603116580	0.85756918340603083273	3.33×10^{-16}

Diagonal Blobs $\varepsilon = 0.80$, min_samples=5, $k = 3$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.72155362533991207474`	`0.72155362533991174168`	3.33×10^{-16}
K-Means	`0.70432533971787614213`	`0.70432533971787558702`	5.55×10^{-16}

5.3 Precision Interpretation

Magnitude	Interpretation
$= 0$	Exact bit-for-bit agreement
$< 10^{-14}$	Normal IEEE 754 rounding; implementations are **numerically equivalent**
$\geq 10^{-12}$	Potential algorithmic discrepancy

5.4 Dataset Visualisations

Each figure shows three panels: **(left)** ground truth; **(centre)** DISCO scores for DBSCAN; **(right)** DISCO scores for K-Means. Colour scale: blue (-1) through white (0) to red ($+1$).

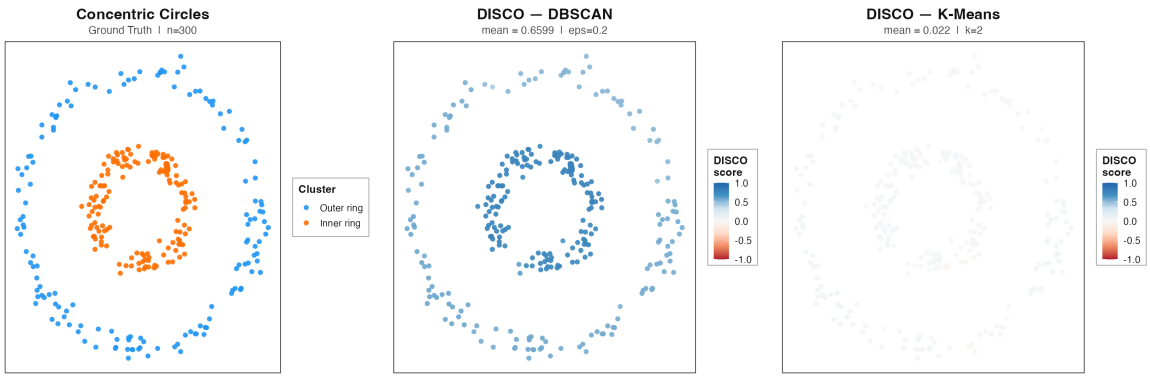

Figure 1. Concentric Circles ($n = 300$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN correctly separates both rings (score 0.690); K-Means cuts through them with a vertical boundary (score 0.027).

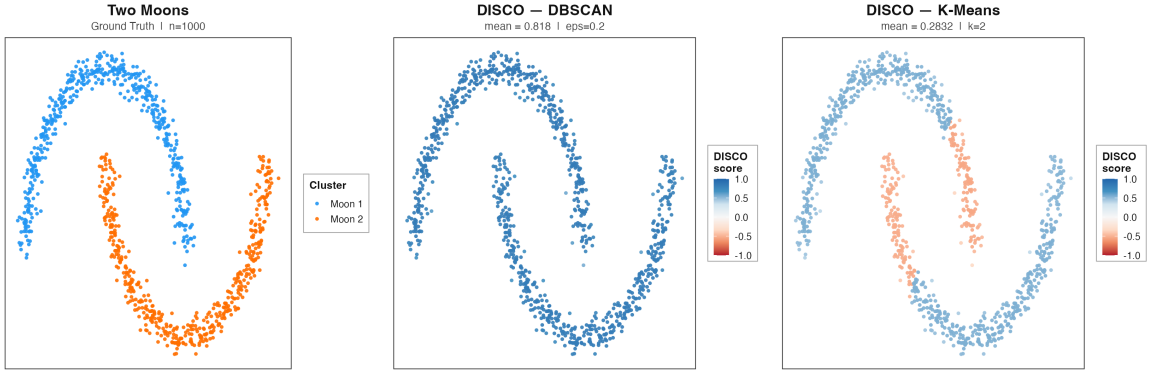

Figure 2. Two Moons ($n = 1000$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN follows both crescents (score 0.818); K-Means splits linearly (score 0.283).

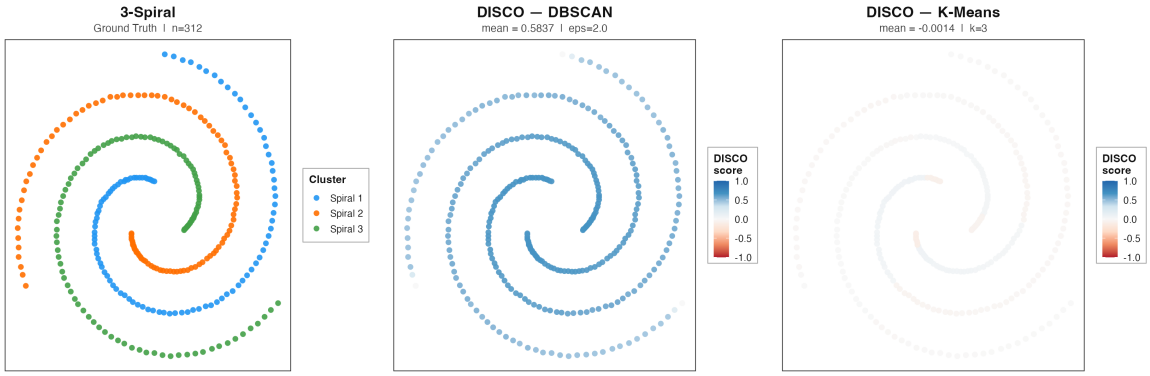

Figure 3. 3-Spiral ($n = 312$, $\varepsilon = 2.0$, $k = 3$, $\mu = 5$). DBSCAN traces the spirals (score 0.584); K-Means actively harms the structure (score -0.0014).

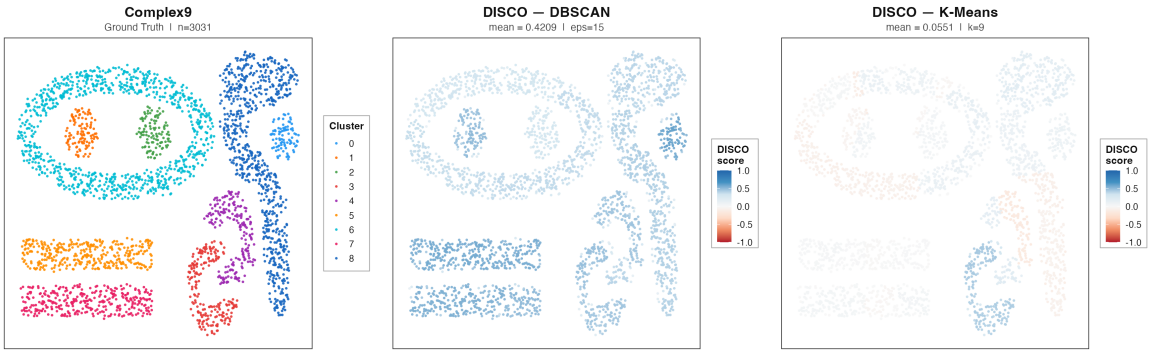

Figure 4. Complex9 ($n = 3031$, $\varepsilon = 15$, $k = 9$, $\mu = 5$). Mixed convex and non-convex clusters. DBSCAN (0.421) outperforms K-Means (0.055).

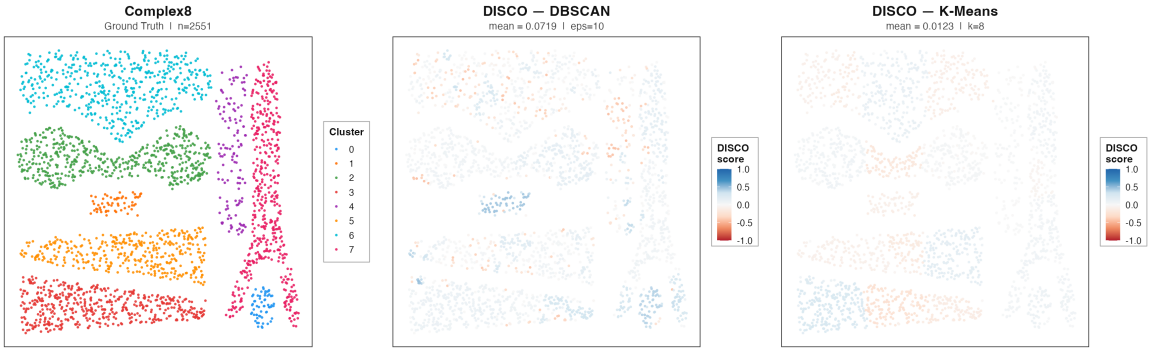

Figure 5. Complex8 ($n = 2551$, $\varepsilon = 10$, $k = 8$, $\mu = 5$). High within-cluster density variation limits both algorithms.

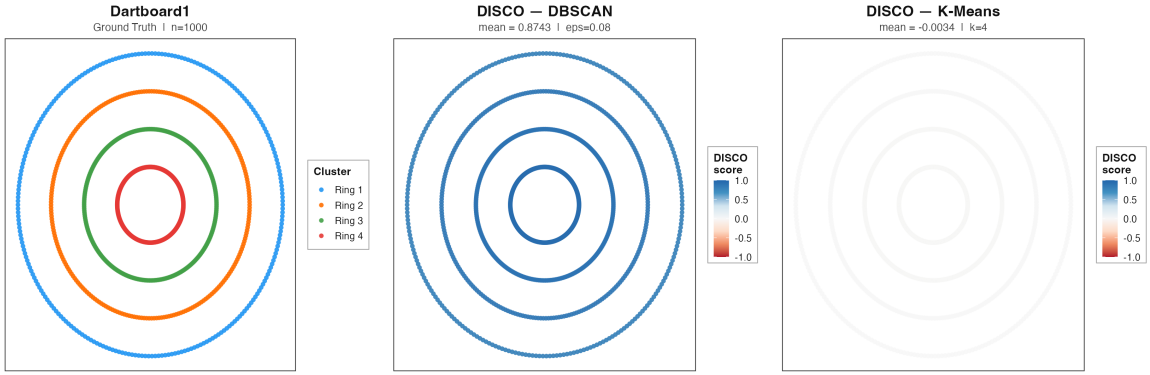

Figure 6. Dartboard1 ($n = 1000$, $\varepsilon = 0.08$, $k = 4$, $\mu = 5$). Four concentric rings: highest DBSCAN score (0.874); K-Means pizza-slice boundary is actively harmful (score -0.003).

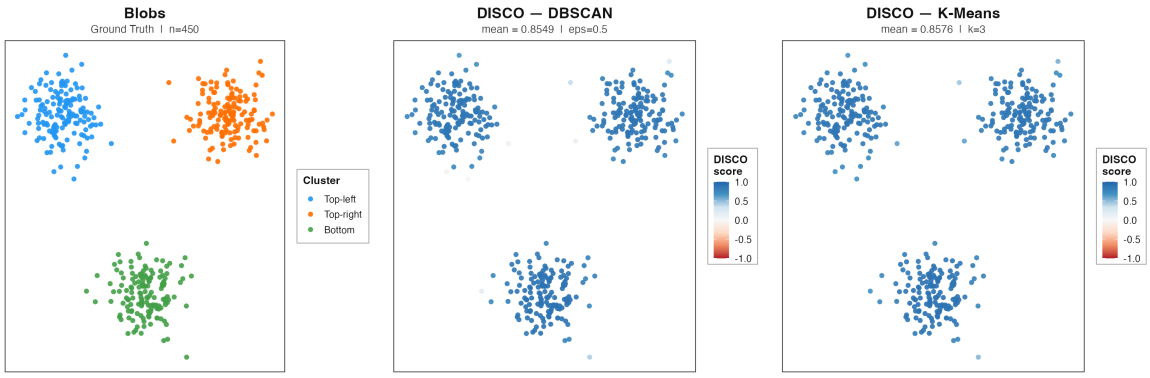

Figure 7. Blobs ($n = 450$, $\varepsilon = 0.5$, $k = 3$, $\mu = 5$). Convex Gaussian clusters: both algorithms score above 0.85, confirming DISCO rewards correct clustering regardless of algorithm type.

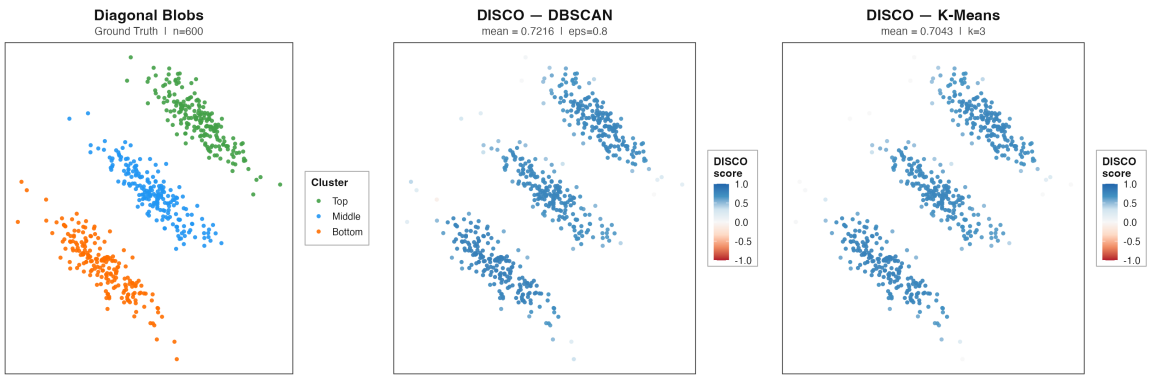

Figure 8. Diagonal Blobs ($n = 600$, $\varepsilon = 0.8$, $k = 3$, $\mu = 5$). Elongated but linearly separable clusters: both algorithms perform well.

5.5 Cross-Dataset Summary

Dataset	DBSCAN	K-Means	<0?	Key observation
Concentric Circles	0.6901	0.0270	—	K-Means cuts through rings
Two Moons	0.8180	0.2832	—	K-Means splits linearly
3-Spiral	0.5837	-0.0014	✓	Spiral impossible for K-Means
Complex9	0.4209	0.0551	—	Mixed shapes favour DBSCAN
Complex8	0.0719	0.0123	—	Density variation limits both
Dartboard1	0.8743	-0.0034	✓	4 rings: K-Means most harmful
Blobs	0.8549	0.8576	—	Convex: both algorithms succeed
Diagonal Blobs	0.7216	0.7043	—	Elongated but separable

Remark 5.1. Negative K-Means scores on Dartboard1 and 3-Spiral are **correct and expected**: K-Means assigns points to clusters further in dc-distance than a neighbouring cluster. Near-identical scores on Blobs confirm that DISCO does not artificially favour density-based algorithms on convex data.

6. API Reference

6.1 `disco_score`

```
disco_score(X, labels, min_points = 5)
```

Argument	Type	Description
`X`	matrix/df	$n \times p$ numeric feature matrix
`labels`	integer vec.	Cluster labels; `-1` for noise
`min_points`	integer	Neighbourhood size (≥ 2). Default: 5

Returns: Single `numeric` in $[-1, 1]$.

6.2 disco_samples

```
disco_samples(X, labels, min_points = 5)
```

Returns: `numeric` vector of length n.

6.3 compute_dc_distances

```
compute_dc_distances(X, min_points = 5)
```

Returns the $n \times n$ DC-distance matrix. Precompute once, reuse:

```
D  <- compute_dc_distances(X, min_points = 5)
s1 <- p_cluster(D, labels1, precomputed_dc_dists = TRUE)
s2 <- p_cluster(D, labels2, precomputed_dc_dists = TRUE)
```

6.4 Utility Functions

Function	Description
`make_circles(n, noise, factor, seed)`	Concentric circles dataset
`make_moons(n, noise, seed)`	Two interleaving crescents
`make_blobs(n, centers, std, seed)`	Gaussian blobs
`plot_disco_scores(X, labels, scores)`	Colour-coded scatter plot
`compare_clusterings(X, list(...))`	Rank multiple clusterings by DISCO
`summary_disco_scores(scores)`	Mean, median, SD, quantiles

7. Score Interpretation

DISCO is stricter than the Silhouette Coefficient: values close to 1 are rare even for good clusterings [Beer et al., 2025]. The following ranges reflect the typical behaviour observed in practice:

Range	Quality	Interpretation
$[0.5,\ 1.0]$	Excellent	Rare; exceptionally well-separated clusters
$[0.3,\ 0.5)$	Good	Clear, well-structured clustering
$[0.1,\ 0.3)$	Moderate	Acceptable; some overlap
$[-0.1,\ 0.1)$	Poor	Weak structure; clusters barely separable
$[-1.0,\ -0.1)$	Very poor	Clustering actively harmful

Note

Common pitfall. `dbscan()` in R uses `0` for noise, not `-1`. Always convert before passing to `discoCVI`:

```
# WRONG
labels <- dbscan(X, eps = 0.2)$cluster
```

```r
# CORRECT
labels <- dbscan(X, eps = 0.2)$cluster - 1L
```

8. Conclusion

The `discoCVI` R package is a faithful, numerically verified translation of the Python DISCO reference implementation [Beer et al., 2025]. Three bugs were identified and corrected — a core-distance off-by-one, an MST tie-breaking discrepancy, and K-Means cross-language non-determinism.

Validation across **eight benchmark datasets** confirms that R and Python produce identical results to within IEEE 754 floating-point precision ($|\Delta| < 10^{-14}$). DISCO correctly discriminates between algorithms: density-based methods outperform centroid-based ones on non-convex data, negative scores meaningfully penalise harmful partitions, and near-equal scores on convex data confirm the metric is unbiased.

The package is available on GitHub and has been submitted to CRAN.

References

Anna Beer, Lena Krieger, Pascal Weber, Martin Ritzert, Ira Assent, and Claudia Plant. Internal evaluation of density-based clusterings with noise. *arXiv preprint arXiv:2503.00127*, 2025. URL https://arxiv.org/abs/2503.00127. Presented at ICLR 2026. https://iclr.cc/virtual/2026/poster/10007920.

David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2):224–227, 1979.

Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.